ktunotes.in

GET IT ON Google Play



KTU NOTES

The learning companion.

KTU STUDY MATERIALS | SYLLABUS | LIVE NOTIFICATIONS | SOLVED QUESTION PAPERS

Website: www.ktunotes.in

CST414
DEEP LEARNING
Module-3 PART -I

Ktunotes.in

SYLLABUS

- **Module-3 (Convolutional Neural Network)**
- **Convolutional Neural Networks – convolution operation, motivation, pooling, Convolution and Pooling as an infinitely strong prior, variants of convolution functions**, structured outputs, data types, efficient convolution algorithms

KRACKEN.in

CONVOLUTIONAL NEURAL NETWORK

- The name "convolutional neural network" indicates that the network employs a mathematical operation called **convolution**. Convolution is a specialized kind of linear operation.
- *Convolutional networks are simply neural networks that use convolution in place of general matrix multiplication in at least one of their layers.*
- **The Convolution Operation**
- Suppose we are tracking the location of a spaceship with a laser sensor. Our laser sensor provides a single output $x(t)$, the position of the spaceship at time t. Both x and t are real-valued, i.e., we can get a different reading from the laser sensor at any instant in time.
- To obtain a less noisy estimate of the spaceship's position, we would like to average together several measurements.

Ktunotes.in

- more recent measurements are more relevant
- We will want this to be a weighted average that gives more weight to recent measurements a weighting function $w(a)$, where a is the age of a measurement
- If we apply such a weighted average operation at every moment, we obtain a new
- function providing s a smoothed estimate of the position of the spaceship

$$s(t) = \int x(a)w(t-a)da$$

Kkuniteki.in

- This operation is called **convolution**. The convolution operation is typically denoted with an asterisk:
- $s(t) = (x * w)(t)$
- w needs to be a valid probability density function, or the output is not a weighted average.
- **convolution is define** for any functions for which the above integral is defined, and may be used for other purposes besides taking weighted averages.
- In convolutional network terminology, the first argument (in this example, the function x) to the convolution is often referred to as the **input** and the second argument (in this example, the function w) as the **kernel**.
- The output is sometimes referred to as the **feature map**.

KRACLER.IO

- when we work with data on a computer, time will be discretized, and our sensor will provide data at regular intervals.
- it might be more realistic to assume that our laser provides a measurement once per second. The time index t can then take on only integer values.
- If we now assume that x and w are defined only on integer t,
- We can define the discrete convolution:

$$s(t) = (x * w)(t) = \sum_{a=-\infty}^{\infty} x(a)w(t-a)$$

- In machine learning applications, the input is usually a multidimensional array of data and the kernel is usually a multidimensional array of parameters that are adapted by the learning algorithm.
- **multidimensional arrays as tensors**
- Because each element of the input and kernel must be explicitly stored separately, we usually assume that these functions are zero everywhere but the finite set of points for which we store the values.
- **can implement the infinite summation as a summation over a finite number of array elements.**
- **use convolutions over more than one axis at a time.**

- example, if we use a two-dimensional image I as our input, we probably also want to use a two-dimensional kernel K

$$S(i,j) = (I * K)(i,j) = \sum_m \sum_n I(m,n)K(i-m, j-n).$$

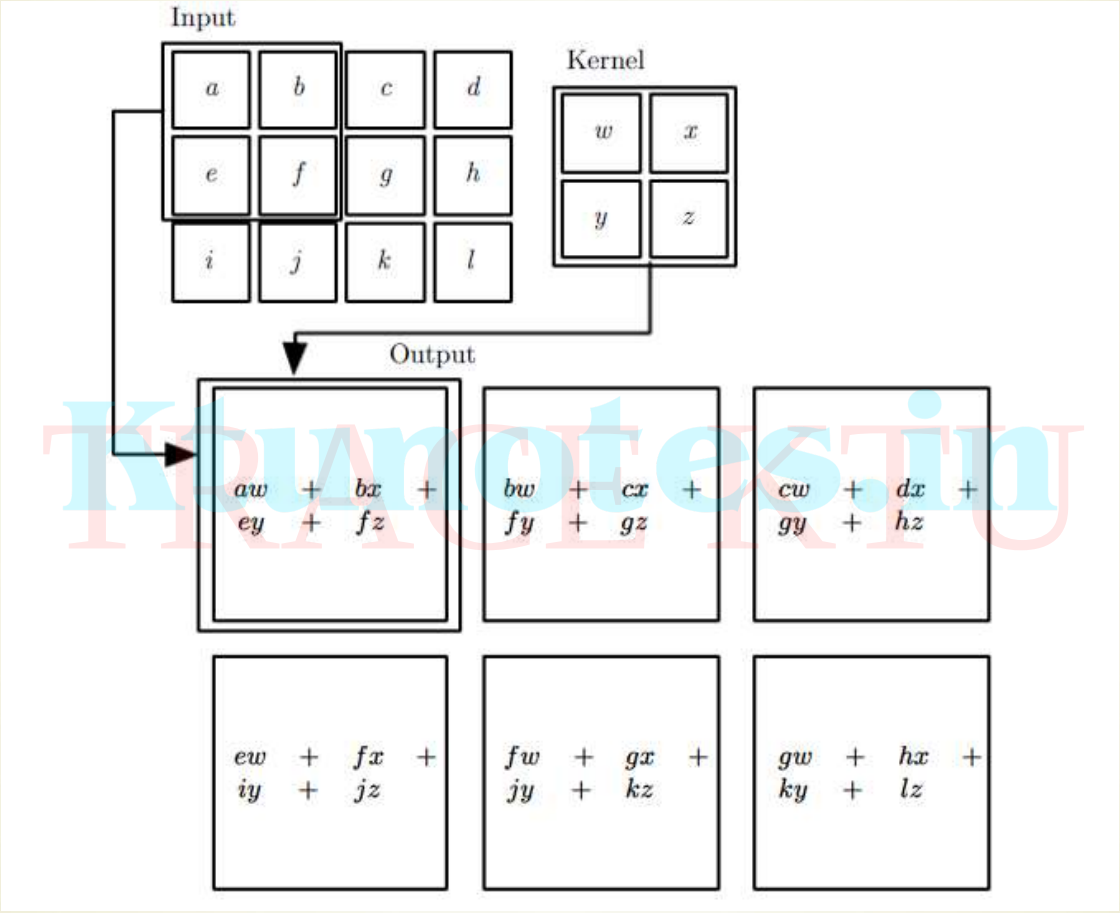
Convolution is commutative, meaning we can equivalently write:

$$S(i,j) = (K * I)(i,j) = \sum_m \sum_n I(i-m, j-n)K(m,n).$$

Ktunotes.in

- ***The commutative property of convolution arises because we have flipped the kernel relative to the input, in the sense that as m increases, the index into the input increases, but the index into the kernel decreases.***
- *The only reason to flip the kernel is to obtain the commutative property.*

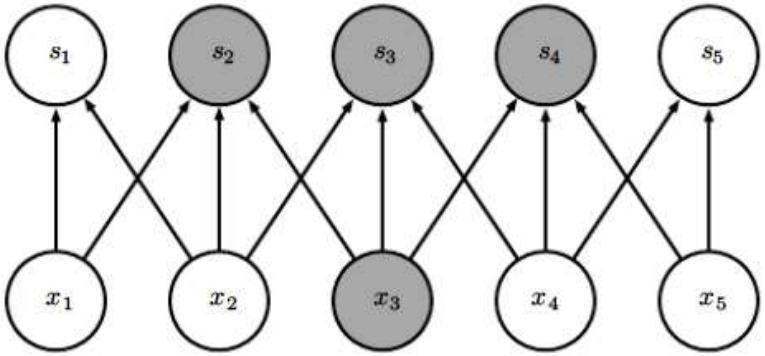
$$S(i,j) = (I * K)(i,j) = \sum_m \sum_n I(i+m, j+n)K(m,n).$$


Input
a b c d
e f g h
i j k l
Kernel
w x
y z
Output
aw + bx + ey + fz
bw + cx + fy + gz
cw + dx + gy + hz
ew + fx + iy + jz
fw + gx + jy + kz
gw + hx + ky + lz

Motivation

- Convolution leverages three important ideas that can help improve a machine learning system: **sparse interactions, parameter sharing and equivariant representations.**
- convolution provides a means for working with inputs of variable size.
- Traditional neural network layers use matrix multiplication by a **matrix of parameters with a separate parameter describing the interaction between each input unit and each output unit**
- This means every output unit interacts with every input unit. Convolutional networks, however, typically have **sparse interactions** (also referred to as **sparse connectivity** or **sparse weights**).
- This is accomplished by making the kernel smaller than the input

Knanet.co.in



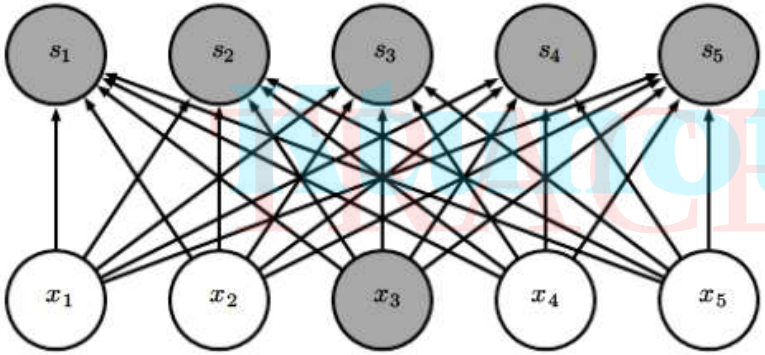


Fig: Sparse connectivity

When s is formed by convolution with a kernel of width 3, only three outputs are affected by x

When s is formed by matrix multiplication, connectivity is no longer sparse, so all of the outputs are affected by $x3$.

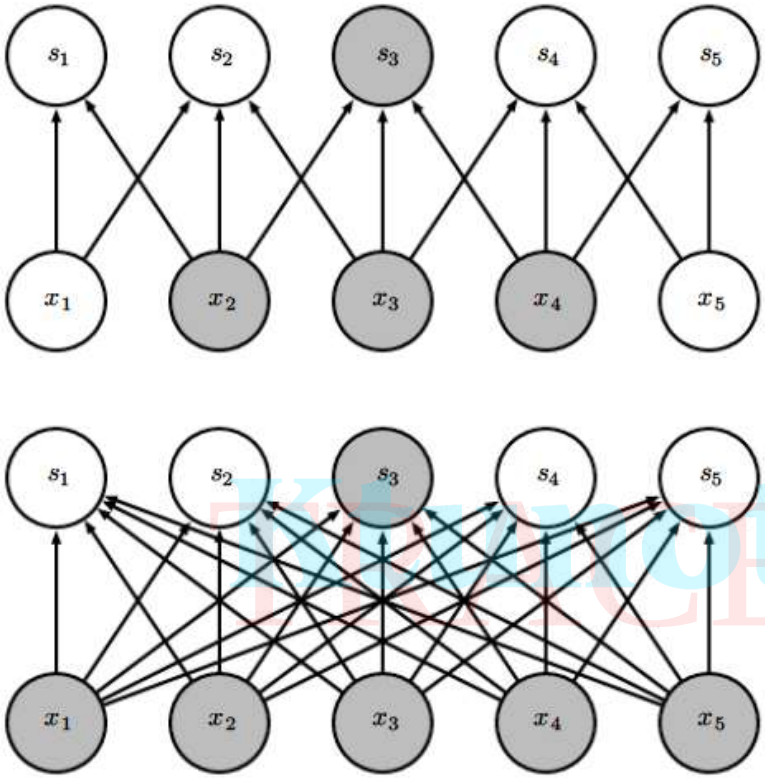
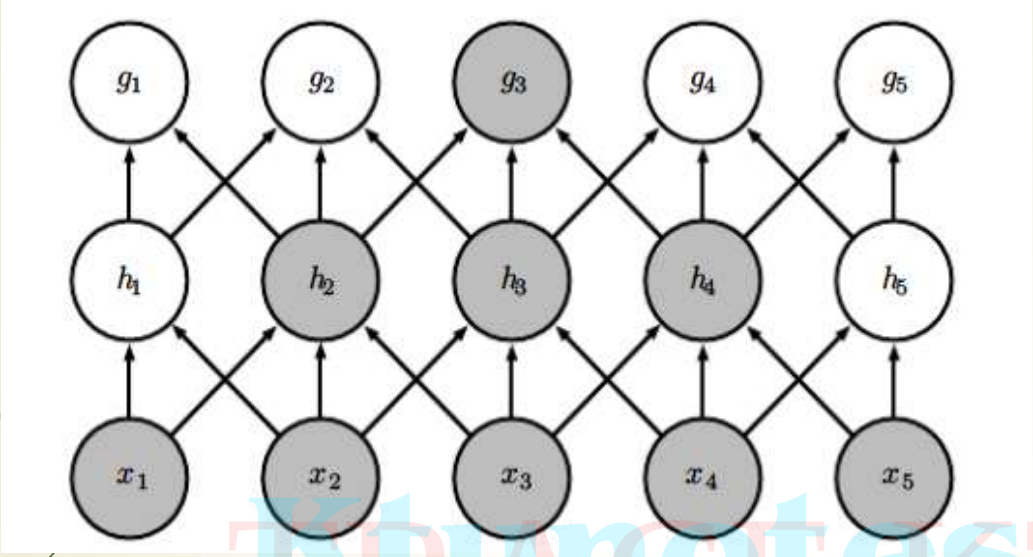


Fig:Sparse connectivity

We highlight one output unit, *s*3, and also highlight the input units in *x* that affect this unit. These units are known as the **receptive field** of *s*3. When *s* is formed by convolution with a kernel of width 3, only three inputs affect *s*3.

When *s* is formed by matrix multiplication, connectivity is no longer sparse, so all of the inputs affect *s*3.



Krnotes.in

Figure :: The receptive field of the units in the deeper layers of a convolutional network is larger than the receptive field of the units in the shallow layers. This effect increases if the network includes architectural features like strided convolution or pooling. This means that even though *direct* connections in a convolutional net are very sparse, units in the deeper layers can be *indirectly* connected to all or most of the input image.

Parameter sharing refers to using the same parameter for more than one function in a model.

- In a traditional neural net, each element of the weight matrix

is used exactly once when computing the output of a layer.

- It is multiplied by one element of the input and then never revisited
- As a synonym for parameter sharing, one can say that a network has **tied weights**, because the value of the weight applied to one input is tied to the value of a weight applied elsewhere.
- In a convolutional neural net, **each member of the kernel is used at every position of the input.**
- The parameter sharing used by the convolution operation means that rather than **learning a separate set of parameters for every location**, we learn only one set

KTUCLASS.IN

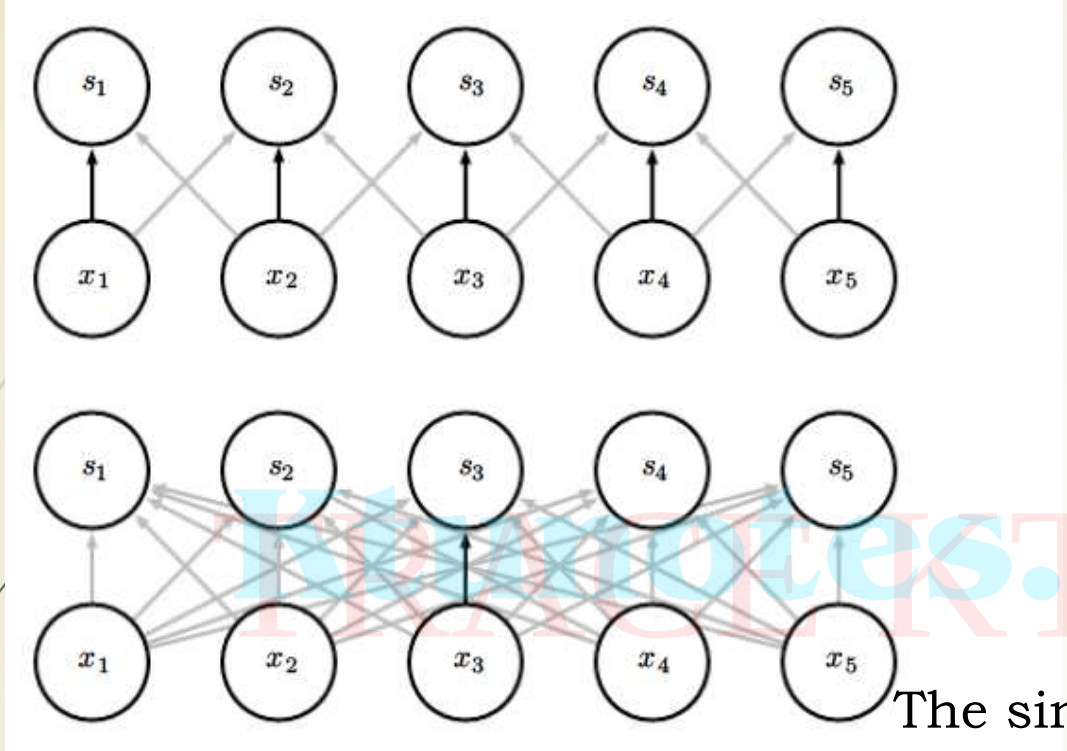


Figure : Parameter sharing

The black arrows indicate uses of the central element of a 3-element kernel in a convolutional model. Due to parameter sharing, this single parameter is used at all input locations.

The single black arrow indicates the use of the central element of the weight matrix in a fully connected model. This model has no parameter sharing so the parameter is used only once.

- In the case of convolution, the particular form of parameter sharing causes the layer to have a property called **equivariance** to translation.
- **Equivariant means that if the input changes, the output changes in the same way.**
- A function $f(x)$ is equivariant to a function g if $f(g(x)) = g(f(x))$
- In the case of convolution, if we let g be any function that translates the input, i.e., shifts it, then the convolution function is equivariant to g.

Ktunotes.in

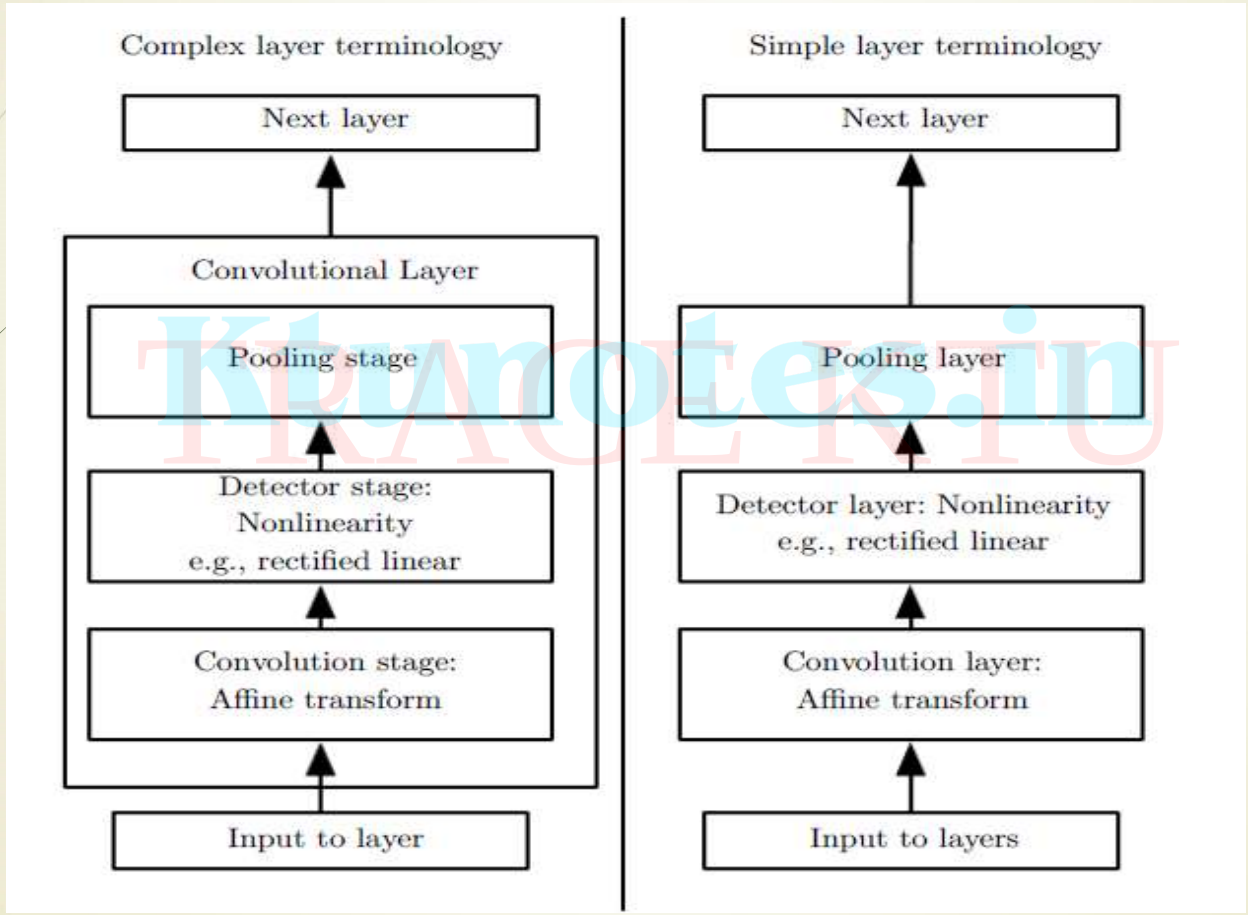
- For example, let I be a function giving image brightness at integer coordinates.
- Let g be a function mapping one image function to another image function, such that $I' = g(I)$ is the image function with $I'(x, y) = I(x - 1, y)$.
- **This shifts every pixel of I one unit to the right.**
- **Finally, some kinds of data cannot be processed** by neural networks defined by matrix multiplication with a fixed-shape matrix.
- Convolution enables processing of some of these kinds of data.

Kvartet.in

POOLING

- typical layer of a convolutional network consists of three stages

The convolutional net is viewed as a small number of relatively complex layers, with each layer having many "stages." there is a one-to-one mapping between kernel tensors and network layers.



The convolutional net is viewed as a larger number of simple layers; every step of processing is regarded as a layer in its own right. This means that not every "layer" has parameters.

- In the **first stage**, the layer performs several convolutions in parallel to produce a set of **linear activations**.
- In the **second stage**, each linear activation is run through a **nonlinear activation function, such as the rectified linear activation function.**
- This stage is sometimes called **the detector stage**. In the *third stage*, we use a pooling function to modify the output of the layer further
- A pooling function replaces the output of the net at a certain location with a summary statistic of the nearby outputs.
- For example, the **max pooling** operation reports the maximum output within a rectangular Neighborhood , popular pooling functions include the average of a rectangular neighborhood

KRAOTES.IN

- In all cases, pooling helps to make the representation become approximately **invariant** to small translations of the input. Invariance to translation means that
- if we translate the input by a small amount, the values of most of the pooled outputs do not change.
- **Invariance to local translation can be a very useful property if we care more about whether some feature is present than exactly where it is.**
- For example, when determining whether an image contains a face, we need not know the location of the eyes with
- pixel-perfect accuracy, we just need to know that there is an eye on the left side of the face and an eye on the right side of the face

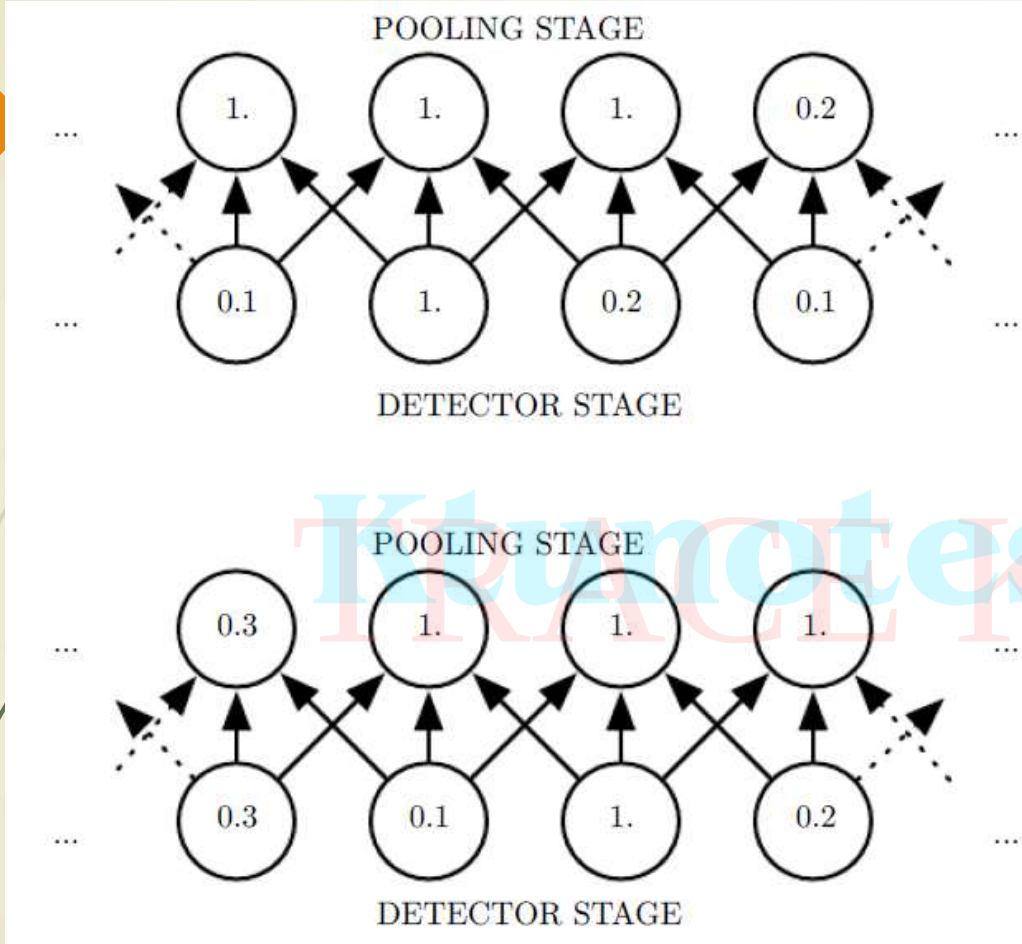


Figure :- Max pooling introduces invariance

A view of the middle of the output of a convolutional layer. The bottom row shows outputs of the nonlinearity. The top row shows the outputs of max pooling, with a stride of one pixel between pooling regions and a pooling region width of three pixels

A view of the same network, after the input has been shifted to the right by one pixel. Every value in the bottom row has changed, but only half of the values in the top row have changed, because the max pooling units are only sensitive to the maximum value in the neighborhood, not its exact location

- The use of pooling can be viewed as adding an infinitely strong prior that the function the layer learns must be invariant to small translations.
- Pooling over spatial regions produces invariance to translation
- Because pooling summarizes the responses over a whole

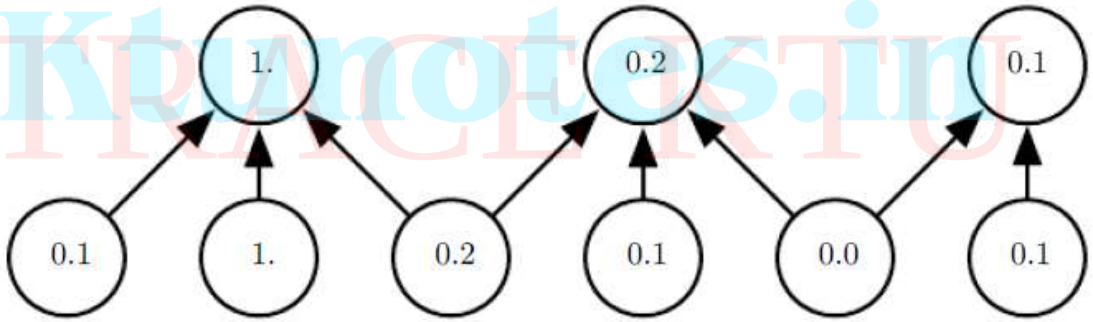
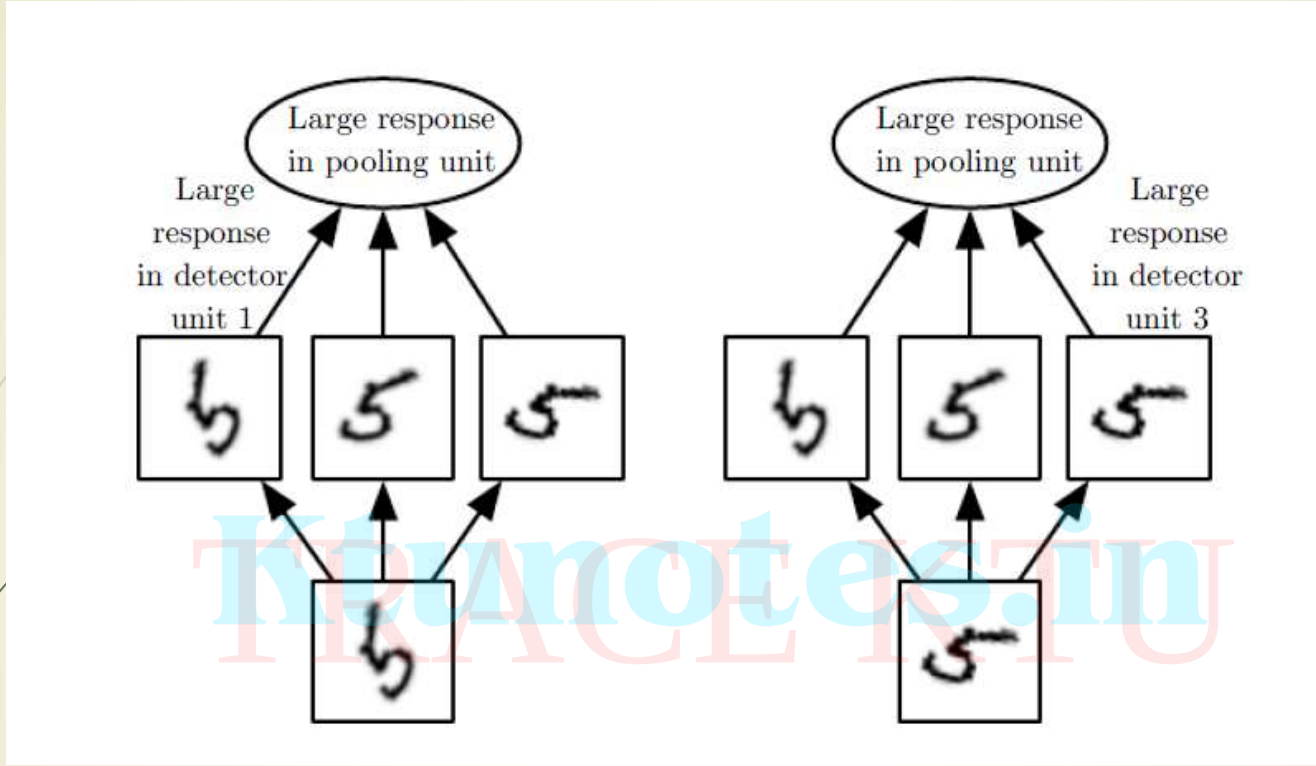


Figure 9.10: *Pooling with downsampling.* Here we use max-pooling with a pool width of three and a stride between pools of two. This reduces the representation size by a factor of two, which reduces the computational and statistical burden on the next layer. Note that the rightmost pooling region has a smaller size, but must be included if we do not want to ignore some of the detector units.



Knacotes.in

Figure : *Example of learned invariances:-*A pooling unit that pools over multiple features that are learned with separate parameters can learn to be invariant to transformations of the input. Here we show how a set of three learned filters and a max pooling unit can learn to become invariant to rotation. All three filters are intended to detect a hand-written 5.Each filter attempts to match a slightly different orientation of the 5. When a 5 appears in the input, the corresponding filter will match it and cause a large activation in a detector unit. The max pooling unit then has a large activation regardless of which detector unit was activated. Max pooling over spatial positions is naturally invariant to translation;

- When the number of parameters in the next layer is a function of its input size
- This reduction in the input size can also result in improved statistical efficiency and reduced memory requirements for storing the parameters
- **For many tasks, pooling is essential for handling inputs of varying size.**

CONVOLUTION AND POOLING AS AN INFINITELY STRONG PRIOR

- **prior probability distribution-**This is a probability distribution over the parameters of a model that encodes our beliefs about what models are reasonable, before we have seen any data.
- Priors can be considered weak or strong depending on how concentrated the probability density in the prior is.
- **A weak prior** is a prior distribution with high entropy, such as a **Gaussian distribution with high variance**
- Such a prior allows the data to move the parameters more or less freely
- **A strong prior** has very low entropy, such as a **Gaussian distribution with low variance**. Such a prior plays a
- more active role in determining where the parameters end up.

KRNOTES.in

- **An infinitely strong prior places zero probability on some parameters and says that these parameter values are completely forbidden, regardless of how much support the data gives to those values**
- This infinitely strong prior says that the but shifted in space. **weights for one hidden unit must be identical to the weights of its neighbor,**
- The prior also says that **the weights must be zero**, except for in the small, spatially contiguous receptive field assigned to that hidden unit.
- Use of convolution as introducing an infinitely strong prior probability distribution over the parameters of a layer.

- Says that the function the layer should learn contains only local interactions and is Equivariant to translation.
- Likewise, **the use of pooling is an infinitely strong prior that each unit should be invariant to small translations**
- Convolutional net as a fully connected net with an infinitely strong prior can give us some insights into how convolutional nets work
- **Convolution and pooling can cause underfitting**
- Convolution and pooling are only useful when the assumptions made by the prior are reasonably accurate.

Kkuanotes.in

- Convolutional network architectures are designed to use pooling on some channels but not on other channels, in order to get both
- Highly invariant features and features that **will not underfit** when the translation invariance prior is incorrect
- Another key insight from this view is that we should only compare convolutional models to other convolutional models in benchmarks of statistical learning performance.

Ktunotes.in



Figure : Examples of architectures for classification with convolutional networks

Variants of the Basic Convolution Function

- Convolution with a single kernel can only extract one kind of feature
- We want each layer of our network to extract many kinds of features, at many locations.
- In a multilayer convolutional network, **the input to the second layer is the output of the first layer, which usually has the output of many different convolutions at each position**
- When working with images, we usually think of the input and output of the convolution as being 3-D tensors,
- With one index into the different channels and two indices into the spatial coordinates of each channel.

KRACLES.IO

- Because convolutional networks usually use **multi-channel convolution, the linear operations they are based on are not guaranteed to be commutative,**
- even if kernel-flipping is used. **These multi-channel operations are only commutative if each operation has the *same number of output channels as input channels*.**
- Assume we have a 4-D kernel tensor **K** with element $K_{i,j,k,l}$ giving the connection strength between a unit in channel i of the output and a unit in channel j of the input, with an offset of k rows and l columns between the output unit and the input unit.

Ktunotes.in

- Assume our input consists of observed data **V** with element *Vi,j,k* giving the value of the input unit within channel *i* at row *j* and column *k* .
- Assume our output consists of **Z** with the same format as **V**. If **Z** is produced by convolving **K** across **V** without flipping **K**, then

$$Z_{i,j,k} = \sum_{l,m,n} V_{l,j+m-1,k+n-1} K_{i,l,m,n}$$

- where the summation over *l, m and n* is over all values for which the tensor indexing operations inside the summation is valid

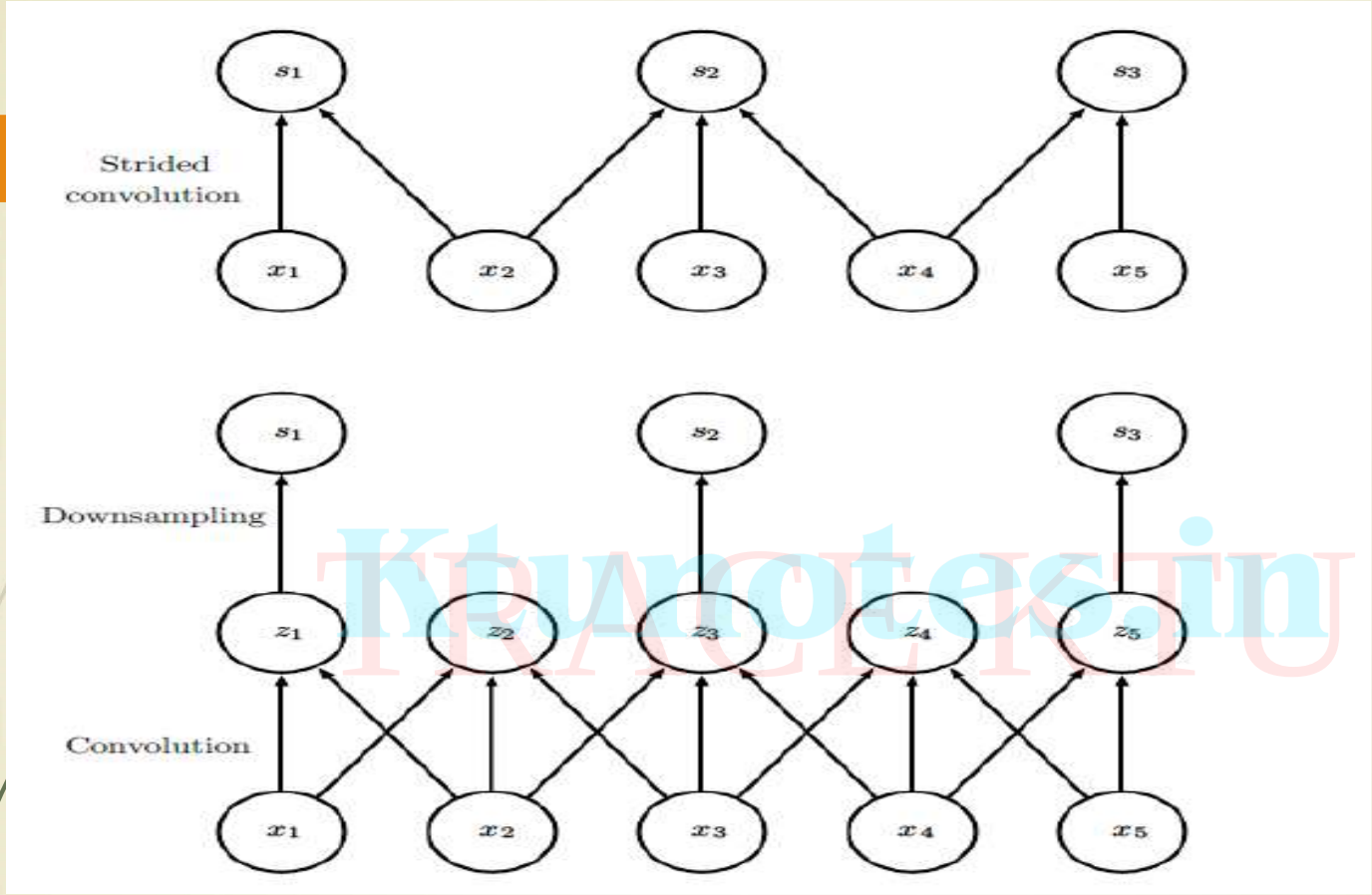
Kvnotes.in

- Want to **skip over some positions of the kernel in order to reduce the computational cost**
- Downsampling the output of the full convolution function.
- If we want to sample only every s pixels in each direction in the output, then we can define a downsampled convolution function c such that

$$Z_{i,j,k} = c(\mathbf{K}, \mathbf{V}, s)_{i,j,k} = \sum_{l,m,n} \left[V_{l,(j-1)\times s+m,(k-1)\times s+n} K_{i,l,m,n} \right].$$

Kurotes.in

- We refer to *s* as the **stride** of this downsampled convolution



Kkuroles.in

Figure : Convolution with a stride. In this example, we use a stride of two.*(Top)*Convolution with a stride length of two implemented in a single operation. *(Bottom)* Convolution with a stride greater than one pixel is mathematically equivalent to convolution with unit stride followed by downsampling. Obviously, the two-step approach involving downsampling is computationally wasteful, because it computes many values that are then discarded.

- One essential feature of any convolutional network implementation is the ability to implicitly zero-pad the input V in order to make it wider
- **Zero padding** the input allows us to control the kernel width and the size of the output independently
- Three special cases of the zero-padding setting
  - the extreme case in which **no zero-padding** is used whatsoever, and the convolution kernel is only allowed to visit positions where the entire kernel is contained entirely within the image.
  - In this case, all pixels in the output are a function of the same number of pixels in the input, so the behavior of an output pixel is somewhat more regular.(In MATLAB terminology, this is called **valid** convolution)

Knowledge.in

- **The size of the output shrinks at each layer**.
- If the input image has width m and the kernel has width k, **the output will be of width $m - k + 1$.**
- Since the shrinkage is greater than 0, it limits the number of convolutional layers that can be included in the network.
- As layers are added, the spatial dimension of the network will
- Eventually drop to 1×1, at which point additional layers cannot meaningfully be considered convolutional

KRUCIBLE.IO

- **Another special case of the zero-padding setting** is when just enough **zero-padding is added to keep the** size of the output equal to the size of the input. (MATLAB calls this **same** convolution)
- In this case, the network can contain as many convolutional layers,
- Since the operation of convolution does not modify the architectural possibilities available to the next layer.
- The **input pixels near the border influence fewer output pixels than the input pixels near the center.**
- This can make the border pixels somewhat underrepresented in the model

Krunotes.in

- Other extreme case, which MATLAB refers to as **full** convolution, **zeroes are added for every pixel to be visited *k* times in each direction**, resulting in an output image of width $M + k - 1$
- **The output pixels near the border are a function of fewer pixels than the output pixels near the center.**
- This can make it difficult to learn a single kernel that performs well at all positions in the convolutional feature map.
- The optimal amount of zero padding (in terms of test set classification accuracy) lies somewhere between "**valid" and "same" convolution**.

Krnotes.in

- In some cases,do not want to use convolution,specified by the 6-D tensor **W**. The indices into **W** are respectively: *i*, the output channel,*j*, the output row, *k*, the output column, *l*, the input channel, *m*, the row offset within the input, and *n*, the column offset within the input.
- The linear part of a locally connected layer is then given by

$$Z_{i,j,k} = \sum_{l,m,n} [V_{l,j+m-1,k+n-1} w_{i,j,k,l,m,n}] .$$

Rknotes.in

- This is sometimes also called **unshared convolution**, because it is a similar operation to discrete convolution with a small kernel, but without sharing parameters across locations
- **Locally connected layers** are useful when we know that each feature should be a function of a small part of space, but there is no reason to think that the same feature should occur across all of space.

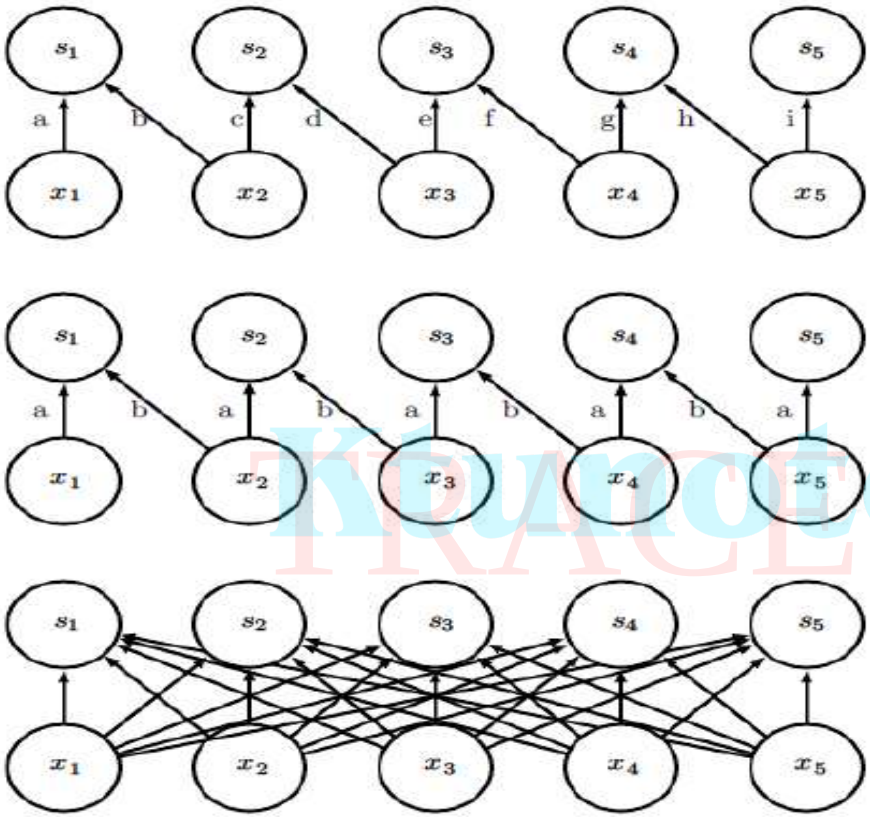


Fig:Comparison of local connections, convolution, and full connections.
*(Top)*A locally connected layer with a patch size of two pixels. Each edge is labeled with a unique letter to show that each edge is associated with its own weight parameter.
*(Center)*A convolutional layer with a kernel width of two pixels. This model has exactly the same connectivity as the locally connected layer. *(Bottom)*A fully connected layer resembles a locally connected layer in the sense that each edge has its own parameter

- **Tiled convolution** offers a compromise between a convolutional layer and a locally connected layer.
- Rather than learning a separate set of weights at *every* spatial location, we learn a set of kernels that we rotate through as we move through space
- This means that **immediately neighboring locations will have different filters,** like in a locally connected layer,
- But the **memory requirements for storing the parameters** will increase only by a factor of the size of this set of kernels, rather than the size of the entire output feature map.

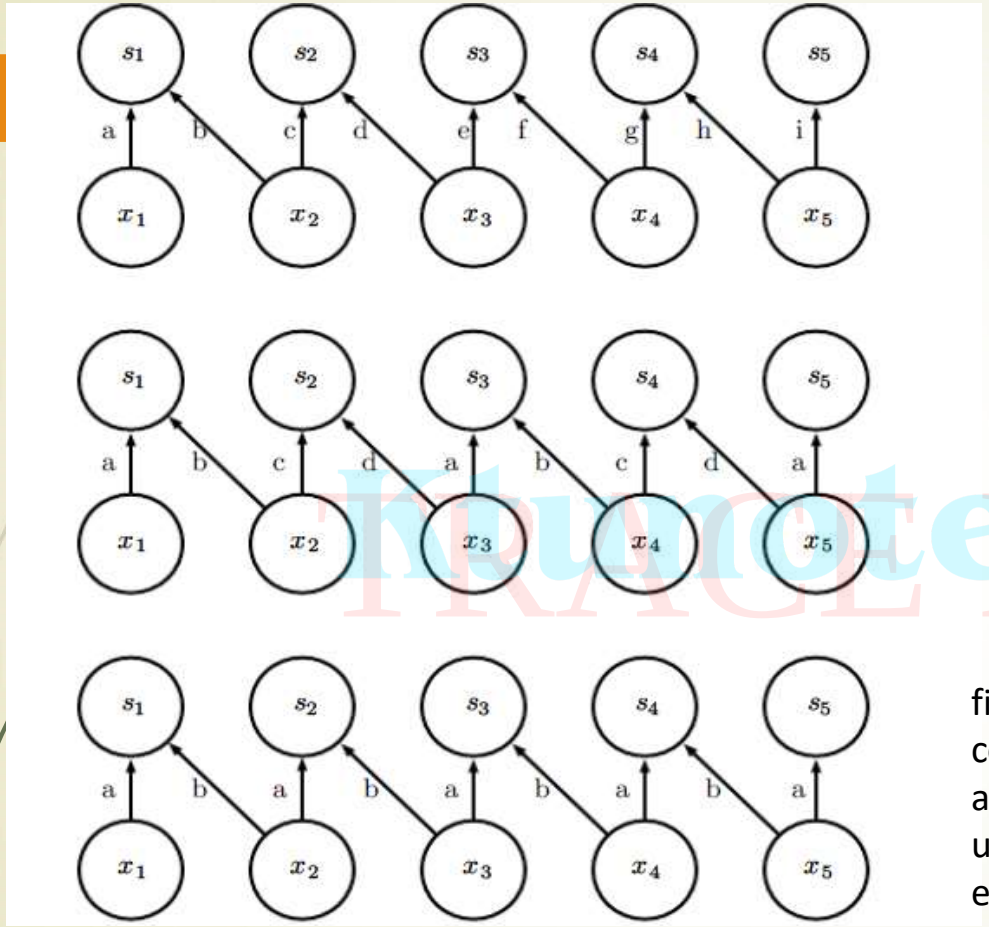


Fig: A locally connected layer has no sharing at all

Fig:-Tiled convolution has a set of *t* different kernels. Here we illustrate the case of *t* = 2. One of these kernels has edges labeled “a” and “b,” while the other has edges labeled “c” and “d.”

fig:-Traditional convolution is equivalent to tiled convolution with *t* = 1. There is only one kernel and it is applied everywhere, as indicated in the diagram by using the kernel with weights labeled “a” and “b” everywhere

fig_A comparison of locally connected layers, tiled convolution, and standard convolution.

- To define tiled convolution algebraically, **let *k* be a 6-D tensor, where two of the dimensions correspond to different locations in the output map**
- Rather than having a separate index for each location in the output map, output locations cycle through a set of t different choices of kernel stack in each direction.
- If t is equal to the output width, this is the same as a locally connected layer.

$$Z_{i,j,k} = \sum_{l,m,n} V_{l,j+m-1,k+n-1} K_{i,l,m,n,j\%t+1,k\%t+1},$$

- where % is the modulo operation, with $t\%t = 0$, $(t + 1)\%t = 1$, etc.
- If these filters learn to detect different transformed versions of the same underlying features, then the max-pooled units become invariant to the learned transformation

Rknotes.in

- To train a convolutional network that incorporates strided convolution of kernel stack **K** applied to multi-channel image **V** with stride *s* as defined by $c(\mathbf{k},\mathbf{v}, s)$
- To minimize some loss function $J(\mathbf{V},\mathbf{K})$. During forward propagation, we will need to use *c* itself to output **Z**, which is then propagated through the rest of the network and used to compute the cost function *J* .
- During back-propagation, we will receive a tensor **G** such that

$$G_{i,j,k} = \frac{\partial}{\partial Z_{i,j,k}} J(\mathbf{V},\mathbf{K}).$$

- To train the network, we need to compute the derivatives with respect to the weights in the kernel. To do so, we can use a function

$$g(\mathbf{G},\mathbf{V},s)_{i,j,k,l} = \frac{\partial}{\partial K_{i,j,k,l}} J(\mathbf{V},\mathbf{K}) = \sum_{m,n} G_{i,m,n} V_{j,(m-1)\times s+k,(n-1)\times s+l}.$$

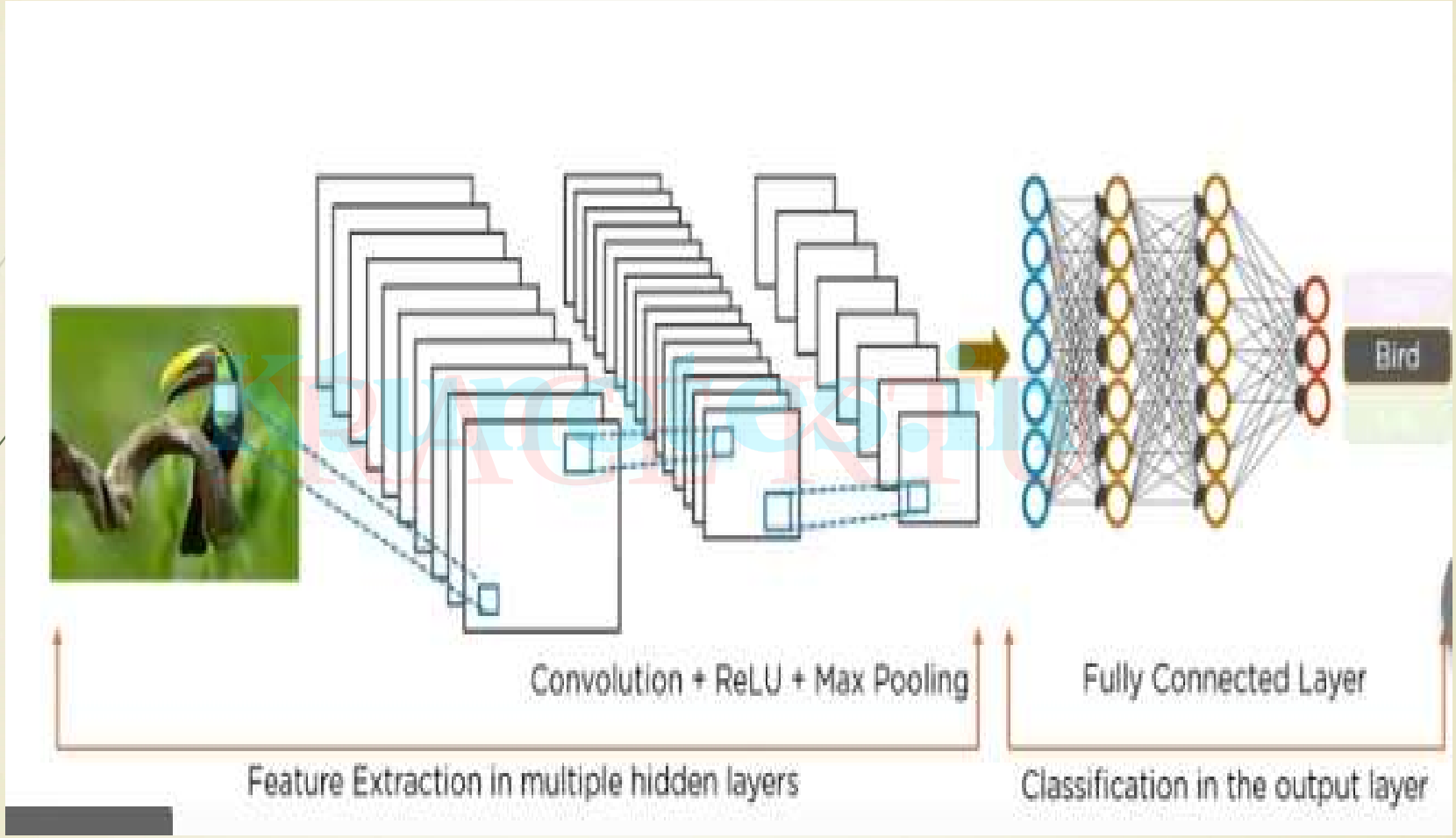
CST414
DEEP LEARNING
Module-3 PART -II

Ktunotes.in

SYLLABUS

- **Module-3 (Convolutional Neural Network)**
- Convolutional Neural Networks – convolution operation, motivation, pooling, Convolution and Pooling as an infinitely strong prior, variants of convolution functions, **structured outputs, data types, efficient convolution algorithms**

KRnotes.in


Bird
Convolution + ReLU + Max Pooling
Fully Connected Layer
Feature Extraction in multiple hidden layers
Classification in the output layer

STRUCTURED OUTPUTS

- Convolutional networks can be **used to output a high-dimensional, structured Object**
- Typically this object is just a tensor, emitted by a standard convolutional layer. For example, the model might emit a tensor **S**, where $S_{i,j,k}$ is the probability that pixel (j, k) of the input to the network belongs to class i.
- This allows the model to label every pixel in an image and draw precise masks that follow the outlines of individual objects.
- Used for classification of a single object in an image, the greatest reduction in the spatial dimensions of the network comes from using pooling layers with large stride.

learncs.in

- One strategy for pixel-wise labeling of images is to produce an initial guess
- of the image labels, then refine this initial guess using the interactions between neighboring pixels.
- Once a prediction for each pixel is made, various methods can be used to further process these predictions in order to obtain a segmentation of the image into regions
- The general idea is to assume that large groups of contiguous pixels tend to be associated with the same label.

Ktunotes.in

DATA TYPES

- The data used with a convolutional network usually **consists of several channels,** each channel being the observation of a different quantity at some point in space or time.
- One advantage to convolutional networks is that **they can also process inputs with varying spatial extents.**
- These kinds of input simply cannot be represented by traditional, matrix multiplication-based neural networks

Knanotes.in

	Single channel	Multi-channel
1-D	Audio waveform: The axis we convolve over corresponds to time. We discretize time and measure the amplitude of the waveform once per time step.	Skeleton animation data: Animations of 3-D computer-rendered characters are generated by altering the pose of a "skeleton" over time. At each point in time, the pose of the character is described by a specification of the angles of each of the joints in the character's skeleton. Each channel in the data we feed to the convolutional model represents the angle about one axis of one joint.

KRAGTeks.id

2-D	Audio data that has been preprocessed with a Fourier transform: We can transform the audio waveform into a 2D tensor with different rows corresponding to different frequencies and different columns corresponding to different points in time. Using convolution in the time makes the model equivariant to shifts in time. Using convolution across the frequency axis makes the model equivariant to frequency, so that the same melody played in a different octave produces the same representation but at a different height in the network's output.	Color image data: One channel contains the red pixels, one the green pixels, and one the blue pixels. The convolution kernel moves over both the horizontal and vertical axes of the image, conferring translation equivariance in both directions.
3-D	Volumetric data: A common source of this kind of data is medical imaging technology, such as CT scans.	Color video data: One axis corresponds to time, one to the height of the video frame, and one to the width of the video frame.

Bharatnotes.in

EFFICIENT CONVOLUTION ALGORITHMS

- Modern convolutional network applications often involve networks **containing more than one million units**. Powerful implementations exploiting parallel computation resources.
- Convolution is equivalent to converting both the input and the kernel to the frequency domain using a Fourier transform, performing point-wise multiplication of the two signals, and converting back to the time domain using an inverse Fourier transform.
- When a d-dimensional kernel can be expressed as the outer product of d vectors, one vector per dimension, the kernel is called **separable**

- **When the kernel is separable, naive convolution is inefficient**. It is equivalent to compose d one dimensional convolutions with each of these vectors.
- The kernel also takes fewer parameters to represent as vectors.
- If the kernel is w elements wide in each dimension, then naive multidimensional convolution requires $O(w^d)$ runtime and parameter storage space, while separable convolution requires $O(w \times d)$ runtime and parameter storage space
- **i) naïve approach,(ii) convolution with separable kernel, (iii) recursive filtering, and (iv) convolution in the frequency domain.**

RVAcoteS.in

Naïve approach

$$h(t) = (f * g)(t) = \int_{-\infty}^{\infty} f(t-\tau)g(\tau)\mathrm{d}\tau.$$

$$h(n) = (f * g)(n) = \sum_{i=-\infty}^{\infty} f(n-i)g(i).$$

Kraglers.in

- The basic (or naïve) approach visits the individual time samples n in the input signal f .
- In each position, it computes inner product of current sample neighbourhood andflipped kernel g, where the size of the neighbourhood is practically equal to the size of the convolution kernel

- Basically, the convolution is a memory-bound problem , i.e. the ratio between the arithmetic operations and memory accesses is low. The adjacent threads process the adjacent signal samples including the common neighbourhood.

Separable convolution

- The convolution with these kernels can be simply decomposed into several lower dimensional convolutions
- Separable convolution kernel must fullfil the condition that its matrix has rank equal to one
- Let us construct such a kernel. Given one row vector
- ~u = (u1, u2, u3, . . . , um) and one column vector ~vT = (v1, v2, v3, . . . , vn). let us convolve them together:

$$\vec{u} * \vec{v} = (u_1, u_2, u_3, \ldots, u_m) * \begin{pmatrix} v_1 \\ v_2 \\ v_3 \\ \vdots \\ v_n \end{pmatrix} = \begin{pmatrix} u_1v_1 & u_2v_1 & u_3v_1 & \ldots & u_mv_1 \\ u_1v_2 & u_2v_2 & u_3v_2 & \ldots & u_mv_2 \\ u_1v_3 & u_2v_3 & u_3v_3 & \ldots & u_mv_3 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ u_1v_n & u_2v_n & u_3v_n & \ldots & u_mv_n \end{pmatrix} = A$$

- It is clear that rank(A) = 1. Here, A is a matrix representing some separable convolution
- kernel while ~u and ~v are the previously referred lower dimensional (cheaper) convolution kernels.

KtuNotes.in

Recursive filtering

- The convolution is a process where the inner product, whose size corresponds to kernel size, is computed again and again in each individual sample. One of the vectors (kernel), that enter this operation, is always the same.

The well-known pure averaging filter in 1D is defined as follows:

$$h(n) = \sum_{i=0}^{n-1} f(n-i) \tag{9}$$

Krnotes.in

The performance of this filter worsen with the width of its support. Fortunately, there exists a recursive version of this filter with constant complexity regardless the size of its support. Such a filter is no more defined via standard convolution but using the recursive formula:

$$h(n) = h(n-1) + f(n) - f(n-n) \tag{10}$$

The transform of standard convolution into a recursive filtering is not a simple task. There are three main issues that should be solved:

1. replication – given slow (but correctly working) non-recursive filter, find its recursive version
2. stability – the recursive formula may cause the computation to diverge
3. accuracy – the recursion may cause the accumulation of small errors

- **Recursive filters on various architectures**
- **Streaming architectures**
- The data can be processed in a stream keeping the memory requirements on a minimum level.
- This allows moving the computation to relatively small and cheap embedded systems.
- The recursive filters are thus used in various real-time applications such as edge detection , video filtering , and optical flow .
- **Parallel architectures**.
- As for the parallel architectures, Robelley et al. presented a mathematical formulation for computing time-invariant recursive filters on general SIMD DSP architectures.

KRACKER.IO

CONVOLUTION IN THE FREQUENCY DOMAIN. (Referred to as the fast convolution.)

- The fast convolution can be even more efficient than the separable version if the number of kernel samples is large enough.

5.1. Fourier transform

The Fourier transform $F = \mathcal{F}[f]$ of a function f and the inverse Fourier transform $f = \mathcal{F}^{-1}[F]$ are defined as follows:

$$F(\omega) \equiv \int_{-\infty}^{+\infty} f(t) e^{-jt\omega} dt, \qquad f(t) \equiv \frac{1}{2\pi} \int_{-\infty}^{+\infty} F(\omega) e^{j\omega t} d\omega. \tag{11}$$

The discrete finite equivalents of the aforementioned transforms are defined as follows:

$$F(k) \equiv \sum_{n=0}^{N-1} f(n) e^{-j(2\pi/N)nk}, \qquad f(n) \equiv \frac{1}{N} \sum_{k=0}^{N-1} F(k) e^{j(2\pi/N)kn} \tag{12}$$

where $k, n = 0, 1, \ldots, N-1$. The so-called normalization factors $\frac{1}{2\pi}$ and $\frac{1}{N}$, respectively, guarantee that the identity $f = \mathcal{F}^{-1}[\mathcal{F}[f]]$ is maintained. The exponential function $e^{-j(2\pi/N)}$ is called the base function. For the sake of simplicity, we will refer to it as W_K.

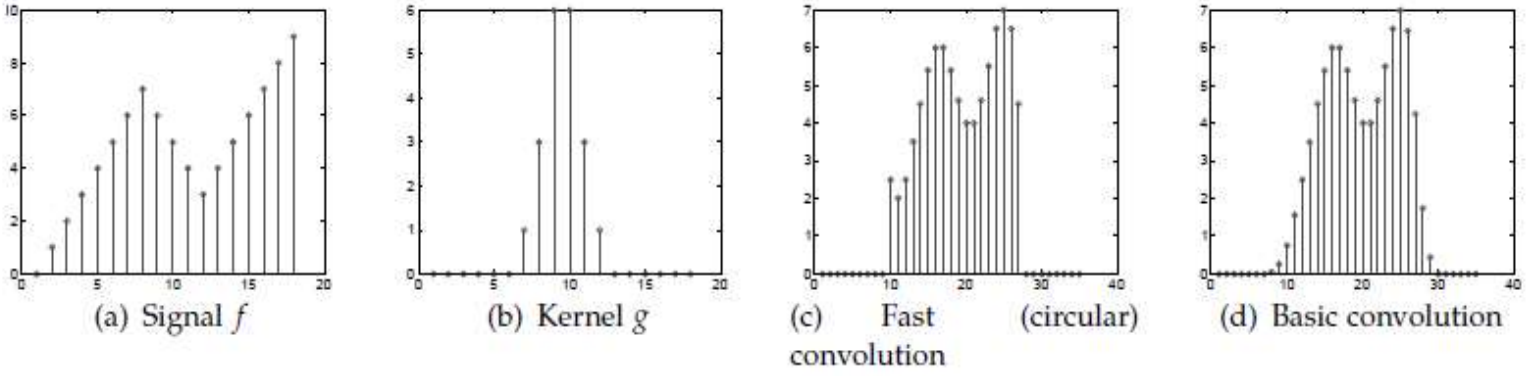


Figure 2. Example of the so-called windowing effect produced by signal f (a) and kernel g (b). The circular convolution causes border effects as seen in (c). The properly computed basic convolution is shown in (d).

IRACE K.in

If the sequence $f(n), n = 0, 1, \ldots, N-1$, is real, the discrete Fourier transform $F(k)$ keeps some specific properties, in particular:

$$F(k) = F(N-k)^*. \tag{13}$$

This means that in the output signal F, only half of the samples are useful, the rest is redundant. As the real signals are typical for many practical applications, in most popular FT and FFT implementations, users are hence provided with special functions to handle real signals in order to save time and memory.